Licence d'Informatique, L2, 2024-2025 – UPVD

Algorithmique des images

Examen de mai 2025

Durée : 2 heures

Tout document moodle autorisé

Pour cet examen sur ordinateur, vous compléterez des fichiers contenant des programmes C d'algorithmes de traitement des images. Vous trouverez sur moodle une archive contenant tous les fichiers nécessaires à la réalisation de l'exercice. En fin d'épreuve, vous déposerez sous moodle **l'ensemble** de votre **répertoire** de travail sous la forme d'**un seul** fichier archive (zip ou tgz).

Attendus

Votre travail complet aura l'organisation suivante.

```
otsu_project/
|
|-- otsu              ← Votre exécutable (à générer avec main.c)
|-- main.c            ← Votre exemple d'utilisation (à compléter)
|-- otsu.h            ← Fichier d'en-tête avec les prototypes
|-- otsu.c            ← Votre implémentation des traitements (à compléter)
|-- pgm.h             ← Fichier d'en-tête de la structure PGM et de fonctions utiles
|-- pgm.c             ← Implémentation de la structure PGM et de fonctions utiles
|-- Makefile          ← Compilation simple
|-- data/             ← Répertoire des fichiers d'entrée et de référence
|-- obj/              ← Vos fichiers objets (à compléter)
|-- res/              ← Vos fichiers de résultats (à compléter)
```

Rappel sur le format d'images PGM

Le format de fichier Portable GrayMap, PGM, permet d'enregistrer des images en niveaux de gris ou chaque pixel est codé par une valeur entière. Ainsi une image de format *pgm* préalablement enregistrée en ASCII ouverte avec un éditeur de texte type 'gedit' permet d'observer la valeur de chaque pixel. Par exemple en ouvrant le fichier `e512.pgm`, on lit :

```
P2    <-- Code pour le format PGM
# Generated with MY_PGM_LIB version 0.0
512 256     <-- Largeur et hauteur de l'image
255         <-- Valeur Maximum
147        <-- pixel 0
```

```
147        <-- pixel 1
149        <-- pixel 2
149        <-- pixel 3
148        <-- pixel 4
148        <-- pixel 6
    ...
```

Les ressources fournies dans `pgm.h` et `pgm.c` définissent la structure PGM et les fonctions utiles au sujet.

Utiliser le `Makefile` fourni.

Seuillage et binarisation d'une image PGM

Le *seuillage* d'image est une technique de binarisation d'image consistant à remplacer les niveaux de gris d'une image par un ensemble de pixels prenant la valeur 255 (blanc) ou 0 (noir) selon que la valeur initiale est inférieure ou supérieure à une valeur définie comme seuil.

Question 1 : `seuillage_image_binaire`

Ecrire la fonction `seuillage_image_binaire` qui prend en paramètres un pointeur sur une structure `pgm`, une valeur de seuil `seuil`, et qui retourne un pointeur sur une structure `pgm` de l'image binarisée correspondante à ce seuil.

Histogramme d'une image PGM

Question 2 : `calcul_histogramme`

Écrire la fonction `histogramme` qui prend en paramètre un pointeur sur une structure `pgm`, un pointeur sur un tableau contenant l'histogramme des valeurs des pixels de l'image et qui retourne une des constantes `EXIT_FAILURE` ou `EXIT_SUCCESS` de `stdlib.h`.

Vérifications

Question 3 : `afficher_histogramme`

Écrire la fonction `afficher_histogramme` qui prend en paramètre un pointeur sur un tableau contenant l'histogramme des valeurs des pixels d'une image, sa taille `max_value`, un indicateur `nom` de l'origine de l'histogramme (par exemple le nom du fichier de l'image associée) et qui affiche dans le terminal les valeurs (x,y) de l'histogramme (un couple de valeurs par ligne).

Question 4 : `sauver_histogramme`

Écrire la fonction `sauver_histogramme` qui prend en paramètre un pointeur sur un tableau contenant l'histogramme des valeurs des pixels d'une image, sa taille `max_value` et qui sauvegarde l'histogramme dans un fichier ASCII `fname`.

Question 5 : `main`

Ecrire `main` qui attend comme seul paramètre un fichier image PGM et qui effectue les traitements suivants.

a. lire le fichier de l'image PGM passé en paramètre
b. effectuer les binarisations par seuillage de cette image pour les valeurs de seuil : 64, 128, 192
c. sauvegarder les trois images ainsi binarisées dans des fichiers PGM
d. calculer les histogrammes de ces trois images binarisées
e. sauvegarder ces histogrammes dans des fichiers ASCII

Question 6 : vérification

- Exécuter `main` sur le fichier `data/e512.pgm`.
- Vérifier votre traitement de la question 5c génère trois images binarisées similaires à celles des trois fichiers références `data/out_64.pgm`, `data/out_128.pgm` et `data/out_192.pgm`.
- Vérifier votre traitement de la question 5e génère trois fichiers identiques aux trois fichiers références `data/hist_64.txt`, `data/hist_128.txt` et `data/hist_192.txt`.

Histogramme normalisé et valeur moyenne

L'histogramme normalisé transforme les valeurs (brutes) de l'histogramme précédent en des valeurs réelles comprises entre 0 et 1. Ainsi écrites, chacune de ces valeurs réelles représente la probabilité d'apparition dans l'image d'un pixel de valeur appartenant à l'intervalle [0 , `max_value`].

Cette normalisation s'effectue pour chaque valeur de pixel $i \in [0, max_value]$ en calculant : $p(i) = h(i)/n$ où n est le nombre total de pixels et $h(i)$ le nombre de pixels de valeur i (ie. la hauteur de la i-ème colonne de l'histogramme brut).

La moyenne totale de l'histogramme normalisé est une caractéristique intéressante. Sachant que $\sum_{i=0}^{max_value} p(i) = 1$, elle est définie comme suit :
$\mu = \sum_{i=0}^{max_value} i \times p(i)$

Question 7 : `histo_normalise_et_mu_total`

Écrire la fonction `histo_normalise_et_mu_total` qui prend en paramètre un pointeur sur une structure `pgm` et qui retourne un pointeur sur un tableau contenant l'histogramme normalisé des valeurs des pixels de l'image et la moyenne totale μ.

Remarque. Ce traitement est indépendant des questions 3 à 6.

Calcul du seuil optimal d'Otsu

Principe de l'approche d'Otsu

A partir de l'histogramme normalisé de l'image, l'algorithme construit une version binarisée et *optimale* de l'image.

Comme avec le seuillage précédent, l'image obtenue ne contient que deux classes de pixels C_1 et C_2 – par exemple le premier plan et l'arrière-plan.

L'algorithme d'Otsu calcule de façon itérative le seuil optimal k, qui sépare au mieux ces deux classes ($0 \leq k \leq$ `max_value`). Ce seuil optimal k maximise la variance inter-classes qui caractérise la dissemblance entre les pixels des deux classes.

Les deux classes C_1 et C_2 obtenues à chaque itération sur k dépendent de k. On les note donc $C_i(k), i = 1, 2$.

Calculs intermédiaires

Les calculs suivants sont les étapes de l'algorithme d'Otsu.

- Calcul de la probabilité $p_i(k)$ qu'un pixel de l'image appartienne à $C_i(k), i = 1, 2$: c-a-d. la somme de toutes les colonnes de l'histogramme normalisé comprises resp. entre 0 et k ou entre $k + 1$ et `max_value`.

- Calcul de la variance inter-classes entre $C_1(k)$ et $C_2(k)$ grâce à l'expression :

$$intervar_{C1(k),C2(k)} = \frac{(\mu \times p_1(k) - \hat{\mu_1}(k))^2}{p_1(k)p_2(k)},$$

où :
- $p_i(k)$ est la probabilité qu'un pixel soit dans $C_i(k), i = 1, 2$,
- μ la moyenne de l'histogramme et
- $\hat{\mu_1}(k) = \mu_1(k)p_1(k) = \sum_{i=0}^{k} i \times p(i)$.

L'algorithme d'Otsu calcule, en itérant sur k, celui qui maximise cette variance inter-classes.

Question 8 : `seuil_otsu`

Écrire la fonction `seuil_otsu` qui prend en paramètres un pointeur sur un tableau de taille `max_value` contenant l'histogramme normalisé des valeurs des pixels d'une image `pgm`, sa moyenne totale `mu_total` et retourne la valeur du seuil optimal de Otsu.

Question 9 : `creer_image_binaire`

Ecrire la fonction `pgm_otsu_binarize` qui prend en paramètre un pointeur sur une structure `pgm` et qui retourne un pointeur sur une structure `pgm` de l'image binarisée correspondante à ce seuil.

Question 10 : `main` (le retour)

Compléter `main` avec les traitements suivants.

a. lire le fichier de l'image PGM passée en paramètre
b. effectuer la binarisation d'Otsu l'image passée en paramètre
c. sauvegarder l'images ainsi binarisée dans un fichier PGM
d. calculer l'histogramme de cette image binarisée
e. sauvegarder l'histogrammes dans un fichier ASCII
f. afficher dans le terminal les histogrammes des quatre images binarisées obtenues.

Question 11 : vérification

Exécuter à nouveau `main` sur le fichier `data/e512.pgm`.

- Vérifier votre traitement de la question 10c génère une image binarisée similaires à celle du fichier référence `data/out_otsu.pgm`.
- Vérifier votre traitement de la question 10e génère un fichier identique au fichier référence `data/hist_otsu.txt`.

Epilogue

Affichage obtenu par la question 11 :

```
Histogramme (out_64.pgm):
Niveau   0 : 7208
Niveau 255 : 123864

Histogramme (out_128.pgm):
Niveau   0 : 44027
Niveau 255 : 87045

Histogramme (out_192.pgm):
Niveau   0 : 121919
Niveau 255 : 9153

Histogramme (out_otsu.pgm):
Niveau   0 : 48751
Niveau 255 : 82321
Seuil optimal d'Otsu = 132.
```

Bonnes vacances.